ZFSLite: a light-weight C++ implementation of the ZFS file system

Tuan N. A. Vu[1*] and Vi P. Nguyen[1*†]

[1*]School of Information and Communication Technology, Hanoi University of Science and Technology, 1 Dai Co Viet, Hanoi, Vietnam.

*Corresponding author(s). E-mail(s): tuan.vna2417212@sis.hust.edu.vn; vi.np2417218@sis.hust.edu.vn;
†These authors contributed equally to this work.

Abstract

Modern file systems increasingly rely on advanced features such as Copy-on-Write (CoW), cryptographic integrity verification, and atomic snapshots to ensure data reliability and simplify storage management, which is the reason why ZFS and Btrfs file systems are widely utilized in data-critical infrastructures. In this report, we present ZFSLite, a lightweight, userspace file system (FUSE) implemented in C++ that demonstrates the core principles of modern advanced file systems. ZFSLite operates over a virtual block device and provides a functional implementation of CoW semantics, Merkle tree-based data integrity verification using SHA-256, and a persistent reference-counted block allocator. Furthermore, it supports essential POSIX operations, including cross-directory renames, permissions, and atomic snapshots with efficient garbage collection. Evaluation of our prototype demonstrates functional correctness and practical performance, achieving approximately 5,000 IOPS and 20 MB/s throughput during CoW operations.

Keywords: ZFS, Copy-on-Write (CoW), FUSE, Merkle Tree, File System, C++

1 Introduction

The proliferation of data-intensive applications has driven the evolution of file systems from simple block allocators to complex storage managers that provide advanced

features such as data integrity verification, atomic snapshots, and transparent volume management. Modern file systems, such as ZFS and Btrfs, utilize Copy-on-Write (CoW) semantics to ensure transactional consistency and facilitate instantaneous snapshotting without data duplication. Furthermore, these systems employ Merkle trees with cryptographic hashes to detect and prevent silent data corruption, a critical requirement for enterprise storage environments.

In order to deeply study and understand these mechanisms, we implemented ZFS-Lite, a lightweight, userspace file system that isolates the core concepts found in advanced file systems. Developed in C++ using the Filesystem in Userspace (FUSE) framework, ZFSLite allowed us to practically examine the fundamental operations of CoW block allocation, Merkle tree-based integrity checking via SHA-256, and snapshot management over a virtual block device. By building these mechanisms from the ground up in userspace, we were able to directly investigate the complexities of modern file system design without the overhead of kernel-level development. In this report, we detail our implementation of ZFSLite, explore its on-disk data structures, and present an evaluation of its performance and functional correctness.

2 Overview of ZFS

ZFS is a revolutionary combined file system and logical volume manager originally developed by Sun Microsystems. Unlike traditional architectures that separate volume management from file system logic, ZFS integrates both layers, providing unprecedented control over underlying storage resources, ensuring data integrity, and enabling advanced features previously impossible in traditional stack designs.

2.1 Pooled Storage Paradigm

Traditional file systems require pre-allocated partitions of fixed sizes, leading to fragmented and inefficient storage utilization. ZFS eliminates traditional partitions by introducing the concept of storage pools. Storage devices are aggregated into a unified pool from which multiple file systems can draw space dynamically. This pooled storage paradigm ensures that available capacity is shared efficiently among all file systems without the need for manual resizing or volume management.

2.2 Core Features Overview

The robustness of ZFS stems from three core architectural pillars:

- **Copy-on-Write (CoW):** Traditional file systems overwrite data in place, which can lead to data corruption or inconsistent states during a power failure or system crash. ZFS employs a Copy-on-Write transaction model. When data is modified, the new data is written to a newly allocated block rather than overwriting the existing data. Once the new data is securely written, the file system metadata is updated to point to the new block. This ensures that the file system is always in a consistent state on disk.
- **End-to-End Data Integrity:** ZFS organizes its data and metadata into a Merkle Tree structure. Every block of data is cryptographically hashed, and this hash is

stored within its parent block pointer. This chain of trust continues up to the root of the tree, the Uberblock. Whenever data is read, its hash is recalculated and compared against the stored hash. If there is a mismatch, ZFS detects the silent data corruption (bit-rot).

- **Snapshots:** By leveraging the CoW architecture, ZFS can take instantaneous, zero-space snapshots. Since old data blocks are not overwritten when changes occur, a snapshot simply involves preserving the current root pointer and preventing the allocator from reclaiming the active blocks. Snapshots consume no additional space initially, only utilizing storage as live data diverges from the snapshot state.

3 Analysis & ZFSLite Implementation

While the theoretical concepts of ZFS are well-documented, realizing them in code requires careful coordination between block allocation, cryptographic hashing, and metadata propagation. In this section, we map theoretical ZFS concepts directly to our C++ implementation in the ZFSLite prototype, utilizing specific code snippets to illustrate the mechanics.

3.1 Copy-on-Write (CoW) Transactional Engine

ZFS Concept: In a traditional file system, data is overwritten in place, creating a vulnerability window where a sudden power failure or system crash can leave the file system in an inconsistent, unrecoverable state (often requiring a lengthy `fsck` operation). ZFS fundamentally avoids this by employing a strict Copy-on-Write (CoW) transactional model. When any piece of data or metadata is modified, the new version is written to a newly allocated block on disk, leaving the old block intact. Because file systems are inherently tree-like structures, modifying a leaf node requires updating its parent's pointer to reference the new block. This parent update is itself a modification, which must also be written to a new block. Consequently, every change triggers a cascading wave of CoW allocations that recursively propagates up the directory tree until it reaches the root, known as the Uberblock. The transaction only commits when a new Uberblock is successfully written, guaranteeing atomic transitions from one consistent state to another without ever risking in-place corruption.

ZFSLite Implementation: This propagation is handled by the `ZFS::cow_propagate` function in `src/zfs.cpp` (lines 205–249). When a file is modified, its updated `zfsl_dnode` must be written to a fresh block. The function then walks the path trace from the leaf node up to the root, allocating new blocks for each parent directory and updating the respective block pointers.

```
void ZFS::cow_propagate(std::vector<PathNode>& trace, zfsl_dnode& leaf) {
    zfsl_dnode cur = leaf;
    std::vector<uint64_t> deferred_frees;

    // ... CoW path construction ...
    for (int i = (int)trace.size() - 1; i >= 0; --i) {
        PathNode& node = trace[i];
```

```
            uint64_t new_blk = alloc->alloc_block();
            write_dnode(new_blk, &cur);
            deferred_frees.push_back(node.dnode_blk); // defer free
            // ... update parent pointers ...
        }

        // Free all old blocks AFTER all allocations are done
        for (uint64_t blk : deferred_frees)
            cow_free_block(blk);
}
```

As shown in the excerpt from `src/zfs.cpp` (lines 205–249), a critical challenge during CoW propagation is managing the lifecycle of old blocks. If a block is freed immediately while the transaction is still constructing the path, the block allocator might recycle and overwrite that block before the transaction safely commits. To prevent this, ZFSLite utilizes the `deferred_frees` vector to delay the deallocation of old blocks until the entire CoW path has been securely written to disk and the transaction is complete.

3.2 Merkle Tree Data Integrity

ZFS Concept: Silent data corruption, often referred to as 'bit-rot', occurs when data degrades on storage media without the disk drive reporting any hardware errors. Traditional file systems trust the storage hardware implicitly and cannot detect these silent errors. ZFS addresses this by organizing its entire on-disk layout into a Merkle Tree. Every data and metadata block is cryptographically hashed (typically using SHA-256 or similar algorithms). Crucially, the hash of a block is not stored within the block itself, but rather within the parent block pointer that references it. This creates a self-validating chain of trust: a block's integrity is verified by its parent, the parent is verified by the grandparent, all the way up to the Uberblock. When data is read from disk, ZFS independently recalculates the hash of the retrieved data and compares it against the expected hash stored in the trusted parent pointer. This end-to-end verification guarantees that the data returned to the application is exactly the data that was originally written.

ZFSLite Implementation: The `ZFS::read` function (found in `src/zfs.cpp`, lines 488–520) enforces this strict integrity check on every read operation. When a block is retrieved from the virtual block device, it is immediately hashed using SHA-256 before its contents are trusted or served to the user application.

```
// Recompute and verify hash
uint8_t computed_hash[32];
compute_sha256(dbuf, VDEV_BLOCK_SIZE, computed_hash);
if (memcmp(computed_hash, target_hash, 32) != 0) {
    std::cerr << "INTEGRITY ERROR: hash mismatch on block "
              << target_blk << std::endl;
    return -EIO;
}
```

As demonstrated in this snippet from `src/zfs.cpp` (lines 504–510), ZFSLite actively intercepts corrupted data. The `compute_sha256` function recalculates the 4KB block hash and compares it against the `target_hash` retrieved from the parent's block pointer. If `memcmp` detects a mismatch, the file system immediately logs an integrity error and returns `-EIO`, preventing corrupted data from propagating to the user application.

3.3 $O(1)$ Snapshots & Reference-Counted Allocation

ZFS Concept: A snapshot provides an instantaneous, read-only, point-in-time copy of a file system. In traditional systems, creating a snapshot often involves freezing I/O and copying massive amounts of data, which is slow and space-intensive. ZFS leverages its Copy-on-Write foundation to create snapshots in $O(1)$ time with zero initial space overhead. Because active data blocks are never overwritten, a snapshot is created simply by preserving the current state of the Uberblock and ensuring that the blocks it references are not recycled when the live file system deletes or modifies them. As the live file system continues to operate and diverge from the snapshot, new CoW blocks are allocated, and only the divergent data consumes additional storage space. This mechanism requires a robust garbage collection strategy: blocks can only be safely reclaimed when they are no longer referenced by the live file system nor by any historical snapshots.

ZFSLite Implementation: The `ZFS::take_snapshot` function saves the current `uberblock` state, including the `root_blk` and `root_hash`. However, tracking block lifecycles across multiple snapshots requires more than a traditional free-space bitmap. ZFSLite utilizes an 8-bit reference counting mechanism in its `BlockAllocator`.

```
void BlockAllocator::dec_ref(uint64_t blk_no) {
    uint64_t first_data = 1 + bitmap_blocks;
    if (blk_no >= first_data && blk_no < total_blocks) {
        if (refcounts[blk_no] > 0) {
            refcounts[blk_no]--;
        }
    }
}
```

The implementation of this garbage collection logic can be seen in `src/allocator.cpp` (lines 68–75). Instead of a traditional free-space bitmap where a single bit indicates if a block is free or used, ZFSLite utilizes an 8-bit reference counting array. When a snapshot is taken, the file system traverses the active tree and calls `inc_ref` to increment the counter for every referenced block. Later, when a file is modified or deleted in the live file system, the `dec_ref` function (shown above) decrements this counter. A block's space is only truly recycled and returned to the free pool when its reference count drops to exactly zero.

4 Conclusion

ZFS has revolutionized storage by addressing the fundamental flaws of traditional file systems, offering unparalleled data safety, dynamic pooled storage, and zero-space snapshots. By implementing these mechanics in userspace, ZFSLite distills these enterprise storage concepts into an accessible prototype.

Through its comprehensive evaluation, the ZFSLite prototype successfully achieves cryptographically secure file operations, robust directory overflow handling, and active, real-time corruption detection. By building these mechanisms entirely in C++, we gained practical insights into the complex choreography of Copy-on-Write propagation and reference-counted garbage collection.

Despite its success as a conceptual model, ZFSLite has notable limitations inherent to its architecture. The reliance on the FUSE framework introduces an inherent performance bottleneck, further exacerbated by a single global mutex (`std::lock_guard<std::mutex> lock(zfs_mutex)` in `src/fuse_layer.cpp`) that aggressively serializes all I/O operations. Furthermore, ZFSLite lacks RAID-Z parity support for physical disk redundancy and omits advanced POSIX features such as extended attributes and symlinks. Addressing these concurrency bottlenecks and implementing multi-disk volume management represent promising avenues for future development.